ArchIntel runtime subsystem: developer handoff

> **Phase 3 runtime update (4 October 2026):** For current session setup, instrumentation preview/exclusions, strict identity declarations, quality diagnostics and recovery, read Phase 3 user flow and developer handoff. Validation results include a new real PetClinic capture; older results below remain historical.

Scope and architectural boundaries

This is an implemented local dynamic-analysis prototype. It accepts actual OTLP traces from the Java agent, preserves sanitized session evidence, maps application classes to completed static snapshots, derives observation coverage, fuses class-pair evidence and exposes runtime-aware review information in the existing application.

It is not a production observability backend or a replacement static engine. Existing static snapshots, metrics, issues, domain assignments and structural scores remain unchanged. New scans add an optional method inventory to class records; old snapshots remain valid and produce unavailable method/entry-point coverage where the denominator is absent.

```
OTel Java agent (external application process)
  -> authenticated session-specific OTLP/HTTP route
  -> bounded wire decode + architectural attribute allowlist
  -> runtime_events (sanitized, deduplicated)
  -> mapping + runtime graph + coverage
  -> static snapshot + runtime graph -> class-pair fusion
  -> flows / persistence signals / readiness / recommendations
  -> runtime and unified APIs -> existing React application
```

The existing jQAssistant/Neo4j pipeline remains responsible for static extraction. Each static scan has an isolated temporary Neo4j store, then PostgreSQL stores the immutable normalized JSON snapshot. Runtime processing uses that snapshot and does not reconnect to or overwrite a static Neo4j store.

Repository structure

Path	Responsibility
`backend/app/runtime/models.py`	RuntimeSession and RuntimeEvent SQLAlchemy persistence.
`backend/app/runtime/schemas.py`	Public Pydantic lifecycle and report envelopes; token hashes are never returned.
`backend/app/runtime/routes.py`	HTTP adapters, bounded request streaming, response transport and application/session selection.
`backend/app/services/runtime/normalization.py`	OTLP protobuf/JSON decoding, gzip bound, ID/time validation, allowlisted normalization.
`backend/app/services/runtime/mapping.py`	Conservative exact, normalized, recognized proxy and unresolved class mapping.
`backend/app/services/runtime/graph_builder.py`	Parent-child reconstruction, aggregation and separate graph-level projection.
`backend/app/services/runtime/coverage.py`	Independent observation metrics and domain states.
`backend/app/services/runtime/fusion.py`	Additive class-pair static/runtime evidence fusion.
`backend/app/services/runtime/intelligence.py`	Request groups, persistence candidates, domain runtime metrics and explainable assessments.

Path	Responsibility
`backend/app/services/runtime/session_service.py`	Session lifecycle, hashing/authentication, ingestion, report selection, retention and agent plan.
`frontend/src/runtime/`	Runtime UI, graph presentation configuration, types and tests.
`scripts/setup-runtime-demo.ps1`	Pinned official agent/sample acquisition and Maven build.
`scripts/validate_runtime_petclinic.py`	Real static + agent + workload + coverage/fusion validation.

This separation keeps parsing and graph algorithms out of routes, prevents runtime state from entering the static scoring services, and gives later worker/queue implementations a service boundary to reuse.

Database changes and lifecycle

Alembic revision **0002** adds two tables; revision 0001 and its static tables are preserved.

`runtime_sessions` links to one application and one completed static analysis. It stores lifecycle timestamps, name/environment, a SHA-256 token hash, span/trace/drop counters, instrumentation settings and an optional completed derived report. `runtime_events` stores session/trace/span identity, receive time and sanitized normalized JSON. A unique constraint on `(session_id, trace_id, span_id)` prevents duplicate ingestion.

Lifecycle: CREATED -> WAITING_FOR_AGENT -> RUNNING -> PROCESSING -> COMPLETED, with FAILED on processing errors. The first accepted span establishes RUNNING and `last_received_at`. A completed session rejects telemetry and cannot be restarted. Stop computes the report before committing COMPLETED. Start/stop are idempotent where appropriate. New tokens revoke previous ingestion credentials.

One process-wide reentrant lock serializes ingestion and lifecycle/report operations. This is intentionally a **single API worker** design. PostgreSQL unique constraints protect identity, but the orchestration lock and counters are not a distributed concurrency protocol. Do not add multiple Uvicorn workers until row locking, durable jobs/leases and atomic bounded admission are implemented.

Running-session evidence is derived on demand. Completed reports are persisted and remain readable after raw event expiry. A background task sweeps every 60 seconds. With the default 24-hour retention setting, active sessions older than 24 hours are completed; sanitized events are deleted 24 hours after completion. The result stores bounded trace/span references, not complete duplicated trace trees. Merging an expired session is rejected; its individual archived report remains readable.

Configuration

Settings are provided through the existing Pydantic configuration and `.env` mechanism. Names correspond to uppercase environment variables.

Setting	Default	Meaning
RUNTIME_PUBLIC_URL	http://127.0.0.1:8000	Address embedded in generated agent instructions.
RUNTIME_MAX_BODY_BYTES	4194304	Both compressed request-body and decompressed payload limit.
RUNTIME_MAX_BATCH_SPANS	5000	Reject oversized normalized batches.
RUNTIME_MAX_SESSION_SPANS	50000	Hard retained-span cap per session; excess returns OTLP partial success.
RUNTIME_RAW_TRACE_RETENTION_HOURS	24	Session age bound and completed-event retention window.

Setting	Default	Meaning
RUNTIME_STRUCTURAL_WEIGHT	0.6	Structural contribution to combined readiness.
RUNTIME_SCORE_WEIGHTS	{"cohesion":0.6,"cycles":0.25,"persistence":0.15}	Runtime component weights; nonnegative and sum to one.

Limits are validated as positive; weights are validated. The configured maximum is a safety bound, not a benchmarked capacity claim. Ten compatible sessions may be selected per report; query-time graph recomputation and JSON payload size need benchmarking before increasing limits.

Agent configuration and protocol

Validated agent: **OpenTelemetry Java agent 2.31.1**. Python wire dependency: `opentelemetry-proto==1.44.0` with its locked protobuf dependency. The Java agent and Python package version numbers need not match; they communicate through OTLP protobuf contracts. Verify protocol compatibility when upgrading.

Use **OTLP/HTTP** with `application/x-protobuf` or `application/json`. Identity and gzip encodings are accepted. gRPC, logs and metrics ingestion are not implemented. Java-agent instructions explicitly select HTTP protobuf and disable metrics/log export. An optional Collector may forward traces to the same authenticated HTTP endpoint; no Collector service is required locally.

OTLP JSON trace/span IDs use hexadecimal. Standard protobuf JSON conversion uses base64 for byte fields, so the binary decoder explicitly converts those IDs to hex before normalization. Both paths validate nonzero trace/span IDs and monotonic positive timestamps. Empty/zero parent IDs represent roots.

The ingestion endpoint is `/api/v1/runtime-sessions/{id}/v1/traces`. `Authorization: Bearer <token>` is checked against that session before body decoding and again inside the ingestion lock. Tokens are generated with 32 bytes of cryptographic randomness and only their hashes are stored. `service.name` is not used as session authorization.

The agent's explicit method-instrumentation configuration comes from the static method inventory. BOUNDARY selects supported annotated application classes; STANDARD additionally selects cycle/high-fan-out classes; DEEP selects broader application classes subject to package filtering. Plans are capped at 200 classes and 100 valid method names per class. Constructor/synthetic names are excluded. Old snapshots without method inventories fall back to available framework spans. No custom bytecode extension or blanket all-JVM-method instrumentation is included.

The live test caught and corrected an invalid `Class[*]` assumption: the verified agent expects explicit method names. Keep a regression test around the generated plan when upgrading the agent.

Normalization and sensitive data

Never persist the original OTLP request. Normalization keeps trace IDs, parent linkage, timestamps/duration, error flags, class/method identifiers, supported HTTP route/method, database system/operation/collection/namespace/server, messaging system/destination, exception type and the small approved attribute set. Attributes must have bounded strings and accepted identifier/route syntax.

Arbitrary span names, resource attributes, event messages, request/response bodies, headers, cookies, SQL text, query parameters and credentials are discarded. Exception messages and stack traces are not retained. The pipeline cannot guarantee an upstream producer has never placed sensitive text in an otherwise allowlisted route or resource name; enterprise deployments need configurable allowlists/redaction and a privacy review. Use route templates, not raw URLs.

The demo JVM's own log file is separate from ingestion and belongs to the trusted PetClinic sample. Customer application execution is never triggered by upload. The real validation script explicitly executes only the downloaded pinned public sample.

Mapping and graph semantics

Static class IDs are FQNs. EXACT requires an exact existing identity. NORMALIZED accepts JVM slash names and `L...;` descriptors. PROXY_RESOLVED only strips recognized Spring CGLIB subclass suffixes when the resulting base class exists exactly. It records a rule and heuristic confidence. JDK proxy numbers, arbitrary generated names and simple-name collisions remain UNRESOLVED; no fuzzy forced match exists.

Unresolved class-bearing spans become `runtime:<identity>` nodes. Spans without class attributes become resource/framework observations with stable IDs and do not count as application classes. Parent reconstruction is keyed by session + trace + span. Cross-trace/session parent IDs cannot create edges. Missing parents are counted; the builder does not skip a gap to invent a direct caller.

Runtime class edges aggregate direct observed nesting by source, target and event type. They store call count, distinct trace count, first/last times, total/average/nearest-rank p95 child span duration, errors and up to 20 source references. These are inclusive child-span duration summaries, not exclusive CPU time. Self-class edges are omitted from the class graph; class-level method coverage still sees their events.

Package/domain projection is separate. Unmapped/resource identities stay in explicit unresolved groups instead of being assigned to business domains. Projected edge counts aggregate calls and bounded references; they do not claim an exact re-aggregated p95 or distinct trace count. Mixed evidence types are listed, not collapsed into a false all-observed claim.

Coverage

Use static class membership and explicit METHOD_CALL class pairs as denominators. Sets deduplicate repeated observations and overlapping selected-session identities. Unresolved/resources cannot inflate class numerators. Zero population is not applicable, never automatic 100%.

The optional static method inventory is extracted by the trusted Java exporter from jQAssistant Method nodes: name, signature and annotation types. Method coverage matches a mapped owner and unique observed method name. Overloads without a reliable signature remain unobserved. Constructors and inherited-reference stubs lacking a parsed method name are excluded from the declared-method denominator. Explicit method annotations identify supported HTTP entry points, scheduled methods and message listeners. Inherited/composed annotations and instrumentation limitations are explained in the metric reason.

Per-domain rows compare observed static class members with all relevant members: UNOBSERVED, PARTIALLY_OBSERVED, OBSERVED. The domain percentage counts domains with at least one observed member. It is not business-scenario or branch completeness. JaCoCo ingestion and instruction/branch coverage are not implemented; add a separate provider contract rather than conflating them with OTLP observation.

Fusion and runtime intelligence

Fusion operates on **directed class pairs**, retaining all supporting static edge types and IDs. Pairs are STATIC_ONLY, DYNAMIC_ONLY or STATIC_AND_DYNAMIC. Therefore a runtime call can corroborate a class relationship without proving that every static field/inheritance/annotation relationship executed. No static edge is deleted because it was not observed.

Request groups are aggregated by entry route/method and participating class set. They include counts, errors, average entry-span duration inputs and bounded trace references. They are not complete serialized call trees. Database evidence uses only directly observed mapped parents; unknown callers/resources remain unknown. Shared persistence candidates require matching database system, server, namespace and resource across mapped domains. No SQL parser guesses table names.

Domain runtime metrics use mapped domain-to-domain/class edges: internal, outgoing/incoming and total calls, error counts, plus observed multi-domain SCC cycle participation. Frequency is evidence, not automatically a defect.

Runtime readiness is `sum(component_score * runtime_score_weight)`. Cohesion is the observed internal-call percentage; cycles score 0 for observed cyclic membership and 100 otherwise; persistence is 25 for a supported shared candidate and 50 for unknown. No mapped calls means runtime score is null. Combined readiness blends structural and runtime scores using the configured structural weight; when

runtime is unknown it retains the structural score. Displayed observation confidence is observed static class fraction, not a calibrated probability.

Low coverage changes confidence/risk messaging and does not subtract from structural readiness. The existing static dependency-first order is retained, with runtime evidence attached for architect review. Runtime-based automatic resequencing, calibrated architecture-style-specific scores and runtime issue taxonomy remain future improvements.

The runtime assistant returns deterministic facts with API evidence links and separate suggested scenarios. It does not send raw telemetry to an LLM. A future generative adapter must preserve bounded evidence retrieval and claim validation.

API surface

All paths below are under `/api/v1` and use existing schema conventions.

Method	Path	Purpose
POST / GET	applications/{id}/runtime-sessions	Create/list sessions; creation returns one-time token/config.
GET	runtime-sessions/{id} or /status	Lifecycle/counters and telemetry-derived agent state.
POST	runtime-sessions/{id}/start or /stop	Begin acceptance or finalize the report.
GET	runtime-sessions/{id}/agent-config	Non-secret configuration.
POST	runtime-sessions/{id}/rotate-token	Revoke old token and generate new connection settings.
POST	runtime-sessions/{id}/v1/traces	Authenticated OTLP HTTP protobuf/JSON ingestion.
GET	runtime-sessions/{id}/evidence or /coverage	Derived report or coverage.
GET	runtime-sessions/{id}/graph/{classes,packages,domains}	Runtime graph by level.
GET	runtime-sessions/{id}/spans/{trace}/{span}	Retained sanitized evidence.
GET	applications/{id}/architecture/unified	Compatible selected-session report via runtime_session_ids.
GET	applications/{id}/architecture/unified/{level}	Selected unified graph projection.
POST	runtime-sessions/{id}/assistant/query	Deterministic bounded runtime explanation.
POST	runtime/retention/purge	Explicit completed-event retention sweep.

Typical errors: 400 malformed/unsupported OTLP, 401 invalid ingestion token, 404 missing/cross-application evidence, 409 invalid lifecycle/baseline/expired merge, 413 body limit, 422 invalid schema or selection. Session cap overflow is OTLP partial success with rejected span count. Duplicate spans are silently deduplicated, not counted as new drops.

Graph projection also accepts `graph_kind=static|runtime|unified`. Static mode returns the unchanged baseline; runtime/unified modes use all compatible selected sessions. The assistant route is intentionally session-specific.

Validation and reproducibility

The baseline before runtime changes was 30 backend and 4 frontend tests. See `RUNTIME-VERIFICATION.md` for final executed checks and `runtime-validation.json` for real measured evidence. Do not infer success from test code alone.

```
.runtime/python/tools/python.exe -m pytest backend/tests -q
$env:ARCHINTEL_RUN_SCANNER_TESTS='1'
.runtime/python/tools/python.exe -m pytest backend/tests/test_scanner_integration.py -q -p no:cacheprovider --basetemp=.data/unique-owned-test-folder
cd frontend
npm test
npm run build
npm exec playwright test
```

The real scanner test directory must be a fresh owned workspace folder because pytest may clean it. Real browser tests need the running API and installed Chrome. `runtime.spec.ts` requires the successfully generated PetClinic validation report.

`setup-runtime-demo.ps1` pins PetClinic revision `30aab0ae764ad845b5eedd76028756835fec771f` (Spring Boot 3.5.0, Java 17 baseline), agent 2.31.1 and Maven Central settings. Maven dependencies are cached in `.runtime/m2`; source/agent downloads are ignored runtime assets. Preserve upstream licenses when redistributing these artifacts. PetClinic test compilation is skipped when preparing this fixture; ArchIntel's own integration assertions validate the product pipeline.

The validation driver runs real jQAssistant extraction, not a mocked scan, and intentionally handles PetClinic's framework-overlapping application namespace without changing default global filters. It stores an authentic static snapshot, runs the JVM on localhost 18080, drives representative HTTP flows, checks coverage increase, all fusion classifications, completed-session behavior and an unchanged static snapshot hash, then terminates only its sample process.

Production work still required

- Authentication/authorization for management APIs and enforced tenant isolation; `tenant_id=local` is not a security boundary.
- Durable distributed ingestion/jobs, transactional admission, cancellation, backpressure, retry semantics and restart recovery for interrupted PROCESSING sessions.
- Rate limits/timeouts per principal, telemetry source/build verification, configurable redaction, encryption and auditable retention.
- Indexed normalized aggregate persistence and measured memory/latency limits; raw JSON recomputation is intentionally simple and bounded.
- Better signature-aware proxy/interface mapping, instrumentation extension only where built-in selected-method tracing is insufficient, and framework/inherited entry inventories.
- Optional code-coverage provider integration; real multi-application/version corpora and overhead benchmarks.
- Calibrated readiness, richer runtime issues/flow isolation, runtime-aware sequencing and evaluated generative explanations.
- Docker runtime verification and production deployment operations. The local Windows path is the demonstrated delivery.

Official contracts used

- OTLP specification: HTTP encodings, ID representation and partial success.
- Java agent configuration: environment configuration and HTTP protobuf transport.
- Method instrumentation: explicit method selection.
- Java agent releases: pinned agent distribution.
- Spring PetClinic source: trusted validation application.